

Reviewer Pack — Minimal Rank of Tropicalized Eichler-Shimura Relations Bound...

Entry d48d4988f736 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 13:46:29 UTC

Minimal Rank of Tropicalized Eichler-Shimura Relations Bounds ACC⁰ Lower Bound

Entry ID: d48d4988f736 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 13:46:29 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Eichler-Shimura Theory) **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

{‘property’: ‘MinimalRank(Eichler-ShimuraRelations(instance)) = $O(\log^c(n))$ for all instances in P with ACC⁰ lower bound’, ‘invariant’: ‘For an explicit function f in P, define the Eichler-Shimura relations associated with its Fourier coefficients. The minimal rank of these relations is bounded by a polynomial in the logarithm of the input size n .’, ‘lower_bound’: ‘There exists a constant $c > 0$ such that for all explicit functions f in P with ACC⁰ lower bound, the minimal rank of the Eichler-Shimura relations associated with its Fourier coefficients is at most $O(\log^c(n))$.’}

Rationale (proposer’s reasoning):

{‘connection’: ‘Eichler-Shimura theory involves the study of modular forms and L-functions, which have connections to number theory. This conjecture would bridge the gap between arithmetic geometry and circuit complexity by connecting properties from Eichler-Shimura theory with ACC⁰ lower bounds.’, ‘explanation’: ‘The use of Eichler-Shimura relations as a complexity invariant is motivated by their connection to modular forms and L-functions, which have been used in other contexts to prove lower bounds. If such a relationship exists for ACC⁰ circuits, it could potentially lead to new techniques for proving lower bounds.’}

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fca6f25103c9691a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all instances in P with ACC^0 lower bound, the minimal rank of the Eichler-Shimura relations associated with its Fourier coefficients is polynomially bounded by $\log(n)$ (i.e., there exists a constant $c > 0$ such that $|\min_rank - \log^c(n)| \leq 3$), and no seed produces a metric greater than $O(\log^c(n))$. The conjecture is falsified if any seed yields a minimal rank exceeding $O(\log^c(n))$ by more than 3 or any seed produces a metric > 10 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - MinimalRank AND Eichler-Shimura Relations AND ACC^0 Lower Bound - Tropicalization AND Eichler-Shimura Theory AND ACC^0 Complexity - Fourier Coefficients AND Eichler-Shimura Theory IN POLYNOMIAL BOUND WITH ACC^0

Top relevant hits considered: - [http://arxiv.org/abs/2003.08644v3] A comparison of positivity in complex and tropical toric geometry - [http://arxiv.org/abs/1510.04604v1] Intersection Theory on Tropicalizations of Toroidal Embeddings - [http://arxiv.org/abs/1308.3813v4] Tropical complexes - [http://arxiv.org/abs/2001.05805v2] Cohomology with integral coefficients of stacks of shtukas - [http://arxiv.org/abs/2004.14244v3] Eulerianity of Fourier coefficients of automorphic forms - [http://arxiv.org/abs/1804.03587v1] Symmetries on plabic graphs and associated polytopes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_explicit_function(n):
        # Placeholder for generating an explicit function with known  $ACC^0$  lower bound
        return [random.randint(1, n) for _ in range(n)]

    def calculate_fourier_coefficients(f):
        # Placeholder for calculating Fourier coefficients of a function
        return [sum(f[i] * math.exp(-2j * math.pi * i * k / len(f)) for i in range(len(f))) / len(f) for k in range(len(f))]

    def eichler_shimura_relations(fourier_coeffs):
```

```

    # Placeholder for calculating Eichler-Shimura relations
    return [sum(fourier_coeffs[i] * fourier_coeffs[j] for i in range(len(fourier_coeffs)) for j in range(i))]

def minimal_rank(relations):
    # Placeholder for calculating the minimal rank of a matrix
    return sum(1 for row in relations if any(row[i] != 0 for i in range(len(row))))

n = random.randint(5, 40)
f = generate_explicit_function(n)
fourier_coeffs = calculate_fourier_coefficients(f)
relations = eichler_shimura_relations(fourier_coeffs)
min_rank_value = minimal_rank(relations)

c = 1.0 # Placeholder for the constant in  $O(\log^c(n))$ 
expected_bound = Fraction(n).log2().limit_denominator() ** c

conjecture_holds = abs(min_rank_value - expected_bound) <= 3
counterexample = f"min_rank={min_rank_value}, expected_bound={expected_bound}" if not conjecture_holds else ""

return {
    "metric_name": "MinimalRank",
    "metric_value": min_rank_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)] # Default to first 5 seeds

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(abs(res["metric_value"] - expected_bound) > 3 or res["metric_value"] > 10 for res in results):
        first_failing_seed = next((seed for seed, res in enumerate(results, start=seeds[0]) if not res["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=min_rank={min_rank_value}, expected_bound={expected_bound}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33ff458c.py", line 62, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33ff458c.py", line 39, in run_trial
    fourier_coeffs = calculate_fourier_coefficients(f)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33ff458c.py", line 27, in calculate_fou
    return [sum(f[i] * math.exp(-2j * math.pi * i * k / len(f)) for i in range(len(f))) / len(f) for k in
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33ff458c.py", line 27, in <genexpr>
    return [sum(f[i] * math.exp(-2j * math.pi * i * k / len(f)) for i in range(len(f))) / len(f) for k in
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: must be real number, not complex

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's validity. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17557
2	preregistration	ollama_remote	glm4:latest	0	0	6597
3	novelty	ollama_remote	glm4:latest	0	0	4865
4	novelty	ollama_remote	glm4:latest	0	0	6026
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14036
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8296
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9198
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9260
9	judge	ollama_remote	glm4:latest	0	0	10426

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 86260 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d48d4988f736.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d48d4988f736.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d48d4988f736.tar.gz` (if generated)